

面向 Android 应用测试的多源感知虚拟环境仿真框架设计与实现

ZhangVirtualEnv 项目开发者

2026 年 8 月

摘要

本项目面向 Android 应用开发、自动化测试和系统兼容性验证中的环境复现问题。实际开发过程中，单独修改位置坐标往往不能解释应用对卫星状态、NMEA、蜂窝小区、WiFi、BLE、SIM 和传感器数据的联合读取，测试结果因此难以复现，也难以判断问题究竟来自应用逻辑、Framework 接口还是厂商 ROM。针对这一问题，我在 ZhangVirtualEnv 中采用控制端、Backend Core、系统测试适配层和版本 Profile 分层的设计，将测试状态集中在统一的环境数据模型中，再根据 Android 版本和 ROM 差异把数据转换为 Framework 对象、Binder 返回值或传感器事件。本文以当前源码和 Android 15/Oplus 系统分析记录为基础，重点讨论路线位置的时间推进与插值、速度和方向计算、GNSS 卫星状态的几何生成、仰角与距离计算、C/N0 平滑模型，以及统一步态相位驱动的加速度、陀螺仪和计步器数据生成。同时，本文说明 CellInfo、WiFi、BLE 和 SIM 测试数据在系统服务边界的构造方式，并分析 SensorService Java 通道与 Native 事件分发通道之间的实际差异。现有实现已经形成单点和路线位置、GNSS 数据路径、CellInfo、WiFi、BLE、SIM 以及多级传感器测试后端；其中 Native 传感器通道是否在目标 ROM 上稳定送达，仍须通过独立检测器、Hook 日志和系统调用结果确认。已有验证表明，软件层测试链路能够提供连续、可观测的多源 API 数据，但它不等同于真实射频 GNSS、真实传感器硬件或物理无线环境。

关键词： Android 环境测试；GNSS 模拟；系统服务适配；LSPosed；传感器仿真；位置服务；ROM 兼容性

目录	2
----	---

目录

一、 引言	4
二、 相关技术与问题分析	4
2.1 Android 感知数据通路	4
2.2 单一位置修改的不足	4
2.3 工程约束	5
三、 系统总体架构设计	5
3.1 状态与数据流	6
3.2 版本 Profile	6
四、 核心算法设计	6
4.1 虚拟轨迹模型	6
4.2 GNSS 卫星状态模型	6
4.3 LLA、ECEF 与 ENU	7
4.4 C/N0 信号模型	7
4.5 统一运动与传感器模型	7
五、 Android 实现方法	7
5.1 Location 与 GNSS 适配	7
5.2 Cellular 与 SIM	8
5.3 WiFi 与 Bluetooth	8
5.4 Sensor Native 适配	9
六、 实验与结果	9
6.1 实验方法	9
6.2 源码与静态证据	10
6.3 已有运行时结论	10
6.4 兼容性结论	10
七、 讨论与限制	10
7.1 软件层边界	10
7.2 厂商 ROM 差异	10
7.3 数据一致性与可观测性	11
7.4 后续方向	11

目录	3
八、 总结	11

一、 引言

移动应用的行为越来越依赖设备所处的综合环境，而不仅是单个位置坐标。地图、运动、物联网和企业应用可能同时读取位置更新、卫星状态、NMEA、CellInfo、WiFi 扫描、BLE 广播、SIM 订阅信息以及加速度和计步事件。对应用开发者而言，真实外场测试成本高、环境难以复现，且 Android 不同版本和厂商 ROM 会改变类名、Binder 参数、服务拆分和事件分发路径。因此，需要一个能够保存测试数据、按时间回放并从系统接口边界提供可控返回值的测试框架。

仅修改 `LocationManager` 或回调中的经纬度并不能形成一致测试环境。应用可以通过 `GnssStatus` 判断卫星是否可见，通过 NMEA 解析另一组坐标，通过 `CellInfo`/`WiFi` 估计网络位置，也可以通过传感器事件推断运动状态。若这些数据仍来自真实设备，应用内部的融合算法会得到相互矛盾的输入。本文将问题定义为：在不修改被测应用源码、也不向第三方应用进程安装测试适配的前提下，如何在 Android 系统服务与 Framework 边界提供可控、可回退、可观测的多源测试数据。

本文贡献如下：

1. 总结 `ZhangVirtualEnv` 的分层架构和环境状态数据流，区分业务引擎、接口适配和版本 Profile 的职责。
2. 从源码给出路线轨迹、GNSS 几何、信号强度和统一运动模型的数学描述。
3. 结合 JADX 对 Android 15 `SensorService` 的结果，说明 Java Runtime Sensor 通道与 Native 分发通道的边界。
4. 基于 `VirEnvDetector` 与既有记录，给出不夸大功能的验证结论和适用限制。

本文将“模拟”限定为软件层测试数据生成与系统 API 适配，不将其表述为真实卫星射频、GNSS 芯片输入或物理网络环境重建。

二、 相关技术与问题分析

2.1 Android 感知数据通路

Android 的位置接口由应用层 `LocationManager`、系统位置服务和 GNSS provider 共同完成；GNSS 状态、NMEA、导航消息和测量数据具有独立的注册与回调链 [1]。蜂窝信息经 `TelephonyManager`、电话 Binder 服务和 `Phone` 对象层返回，`CellIdentity` 与 `CellSignalStrength` 共同构成可供上层解析的结构。WiFi 扫描结果由 WiFi 服务封装为 `ScanResult`，BLE 扫描则经蓝牙 Binder 与蓝牙栈回调 `ScanResult`。传感器事件通常经过 `SensorService` 的 native 事件队列，再进入应用的 `SensorEventListener` [2]。

2.2 单一位置修改的不足

设应用接收到的位置为 L ，但同时观测到的卫星状态、无线环境和运动数据仍为真实值 G, W, M 。应用的融合结果可表示为

$$\hat{P} = F(L, G, W, M).$$

仅修改 L 而保持 G, W, M 不变, 会使 F 在不同数据源之间产生残差。例如, 虚拟位置的 NMEA 坐标与真实卫星状态不一致, 或虚拟速度与真实加速度、步频不一致。框架因此采用统一 Backend 状态, 并让各适配器从同一测试 Profile 或时间轴读取数据。

2.3 工程约束

系统服务 Hook 具有高风险: 错误的反射签名、错误的构造器字段或不兼容的 ROM 偏移可能影响 `system_server`。项目采用候选类/方法匹配、版本 Profile、异常隔离和 fail-open 策略; 适配失败时继续原始调用。`scope.list` 只包含必要系统组件、项目自身控制端和独立检测器, 不把第三方应用作为实现依赖。

三、 系统总体架构设计

我在实现过程中没有把各类 Hook 写成彼此独立的功能集合, 而是先确定了一条统一的数据链路: 控制端只负责提出测试场景, Backend Core 负责保存状态并推进时间, 系统测试适配层负责把状态翻译成 Android 能够理解的接口数据。这样做的直接好处是, 位置、卫星、无线环境和运动数据可以共享同一个时间基准, 某一个 ROM 的类名或方法签名变化也不会反过来污染核心模型。系统结构如图 1 所示。

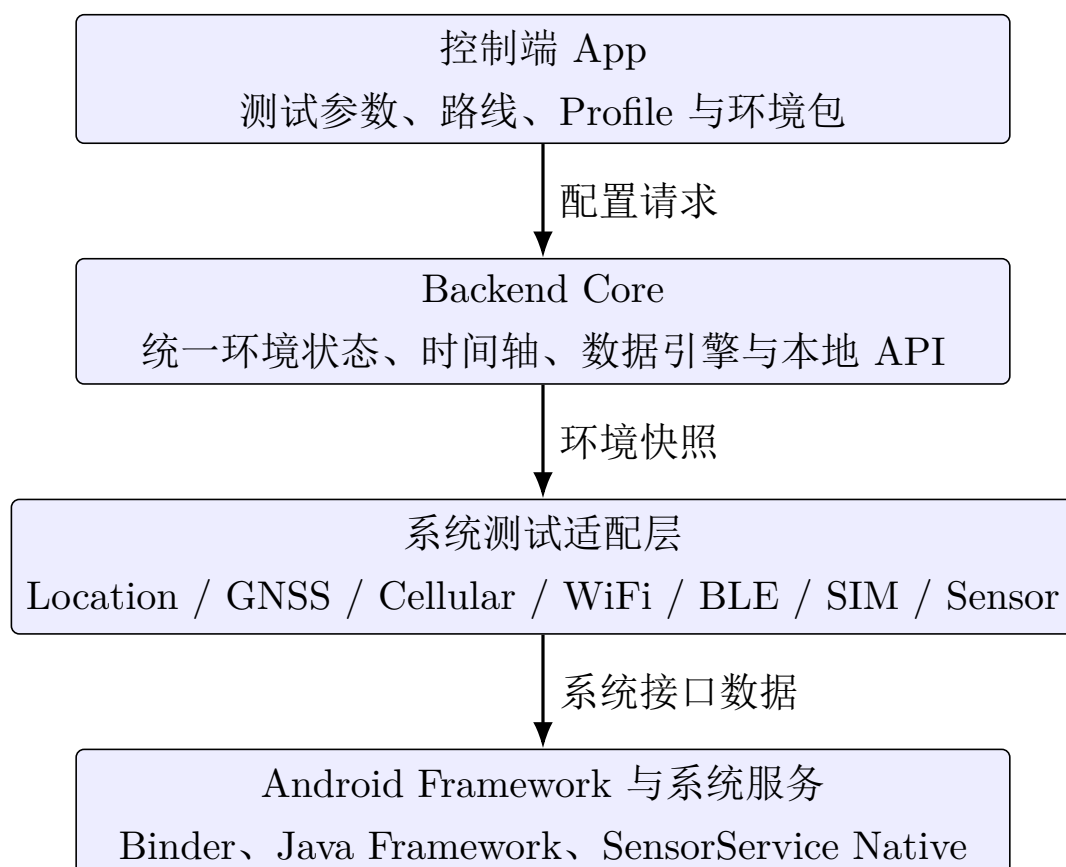


图 1: 系统总体架构与数据边界

3.1 状态与数据流

配置请求沿“控制端 → 本地 API → Backend → 引擎”流动。适配器按周期或回调请求当前快照，生成具有当前时间戳的数据对象。位置是 GNSS 与部分环境生成器的共同参考，但 GNSS 引擎不持有第二份位置状态；路线引擎和单点引擎是位置的唯一所有者。环境采集器则把位置、CellInfo、WiFi、BLE、GNSS 和传感器事件按时间戳写入环境包，回放时由时间轴统一选择帧。

3.2 版本 Profile

Profile 按 SDK、设备指纹和 ROM 差异选择候选类名、方法名、参数数量和边界策略。Android 15 的蓝牙扫描入口与 Android 16 的 `ScanController.startScan` 存在差异，项目通过版本门控保持已验证版本行为不变。未匹配到目标类或方法时，适配器跳过该能力并放行系统原始路径，而不是以默认值覆盖未知 ROM。

四、 核心算法设计

4.1 虚拟轨迹模型

我把位置生成分成单点和路线两种模式，但两者都只维护一份位置状态。单点模式保存经纬度、速度、方向和更新时间；路线模式保存折线点列、段索引和段内进度。对相邻点 $P_i = (\phi_i, \lambda_i)$ 与 P_{i+1} ，先用 Haversine 近似计算段长

$$d_i = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_i \cos \phi_{i+1} \sin^2 \frac{\Delta\lambda}{2}}.$$

在时间间隔 Δt 内按 $v\Delta t$ 消耗路段长度；若未跨段，进度更新为 $q' = q + v\Delta t/d_i$ ，位置以线性插值给出

$$P(t) = (1 - q)P_i + qP_{i+1}.$$

路线引擎采用惰性推进：只有收到 `currentLocation()` 请求时才根据 monotonic 时钟推进，因此没有请求时不会由后台线程产生无观测数据。方向由相邻点的球面方位角计算，速度直接写入 `Location.speed`。循环路线可直接跳回起点，也可沿终点到起点的连线平滑返回。可选的小幅抖动只作用于输出坐标，不改变轨迹游标。

4.2 GNSS 卫星状态模型

GNSS 引擎以位置快照和时间戳为输入，对固定卫星目录逐颗计算状态：

$$S_k = \{\text{SVID}, \text{constellation}, \mathbf{r}_k, e_k, a_k, \rho_k, C/N_{0,k}, u_k\}.$$

目录包含 GPS、BeiDou、Galileo、GLONASS 和 QZSS 测试卫星及主测试频段。轨道部分使用连续时间驱动的圆轨道 Kepler 近似。对模型参数 θ_k ，相位为

$$\alpha_k(t) = \alpha_{0,k} + 2\pi(t \bmod T_k)/T_k,$$

再按倾角和升交点将轨道平面坐标旋转到 ECEF。该算法提供 API 数据路径的连续几何，不包含星历、伪距、导航电文或射频波形。

4.3 LLA、ECEF 与 ENU

观测点的 WGS-84 LLA [5] 为 (φ, λ, h) 。令长半轴为 a 、第一偏心率平方为 e^2 ，曲率半径

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}.$$

则

$$\mathbf{r}_{ecef} = \begin{bmatrix} (N + h) \cos \varphi \cos \lambda \\ (N + h) \cos \varphi \sin \lambda \\ (N(1 - e^2) + h) \sin \varphi \end{bmatrix}.$$

卫星与观测点的 ECEF 差向量经过局部 ENU 旋转得到 (E, N, U) 。距离、仰角和方位角分别为

$$\rho = \sqrt{E^2 + N^2 + U^2}, \quad e = \text{atan2}(U, \sqrt{E^2 + N^2}),$$

$$a = \left(\text{atan2}(E, N) \frac{180}{\pi} + 360 \right) \bmod 360.$$

源码将仰角不低于 5° 的卫星视为可见，并要求仰角不低于 15° 且 $C/N_0 \geq 25$ dB-Hz 才标记为 UsedInFix。不可见行保留在诊断 JSON 中，但不伪造为公开 GnssStatus 行。

4.4 C/N0 信号模型

信号模型采用基准项、仰角增益、距离损耗和慢变化项：

$$C/N_0 = \text{clip}(C_0 + G(e) - L(\rho) + n(t, k), 18, 52).$$

其中 $G(e)$ 在低仰角为负、在高仰角趋于正增益，距离损耗为

$$L(\rho) = 3 \log_2 (\max(\rho/r_k, 0.5)),$$

慢变化项由时间和卫星相位的正弦函数生成，而不是每次刷新独立抽样。这样可以保持 SVID、数量和几何连续，适用于解析器和状态机回归。

4.5 统一运动与传感器模型

VirtualMotionEngine 以单一 monotonic 时钟推进全局步态相位：

$$p_{t+\Delta t} = p_t + \frac{f\Delta t}{60} \pmod{1},$$

其中 f 为 steps/min。相位跨越 1 时，步数增加 1、距离增加一个步长，并产生一个待消费的 STEP_DETECTOR 事件。四段步态波形模拟着地冲击、保持、摆动和恢复；加速度、重力、线性加速度、陀螺仪和计步器都由同一状态生成。手机姿态用低频 pitch/roll 漂移叠加步频摆臂，保证不同传感器之间具有时间相关性。Native C 实现复用了相同的相位、步数和姿态思想，并在传感器事件分发前重写数据。

五、 Android 实现方法

5.1 Location 与 GNSS 适配

Location 适配器在系统位置服务边界接收注册、上报和监听器相关调用，并从 Backend 取得当前测试位置。GNSS 适配器覆盖状态和 NMEA 注册及注销路径；启用测试位置时，不

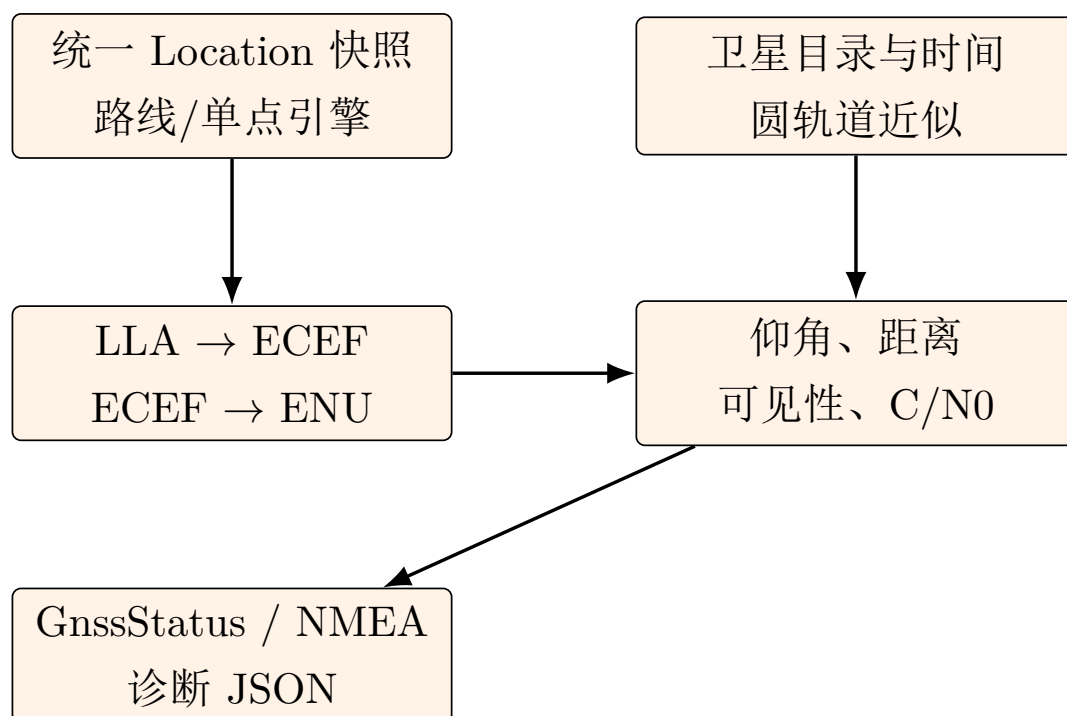


图 2: GNSS 状态生成过程

注册真实回调，而是周期投递由同一位置快照生成的 `GnssStatus` 和 NMEA。导航消息与原始测量属于更低层数据面，当前实现对其采用阻断或预留策略，关闭测试时恢复原始路径。`GnssLocationProvider.onReportSvStatus(GnssStatus)` 与 `onReportNmea(long,String)` 同时用于观察真实系统数据和替换测试数据，所有异常均回退到原始调用。

5.2 Cellular 与 SIM

`CellInfo` 测试数据按 LTE、NR、GSM、WCDMA 和 CDMA 组织。构造器使用 Profile 提供的 MCC/MNC、TAC/LAC、CI/NCI、PCI、ARFCN 和信号强度，并对字段位宽和不可用哨兵进行归一化。Android 15 的 MCC/MNC 字符串字段与旧 int getter 存在差异，读取器优先使用字符串并安全解析。CDMA 位置字段需要按定点量化方式写入 `CellIdentity`，故其是特定 ROM 的兼容构造路径，而不是通用蜂窝定位模型。

SIM 适配位于电话 Binder 服务端和 Phone 对象层，覆盖订阅、运营商、设备标识、号码、网络类型、SIM 状态和信号强度等返回值。实现对无后缀、`ForSubscriber`、`WithFeature` 等方法名建立候选集合；目标不存在或返回值构造失败时调用原始方法。该设计强调接口抽象和兼容性测试，不表示对真实基带或 SIM 卡物理状态的重建。

5.3 WiFi 与 Bluetooth

WiFi 服务由 `ServiceManager` 动态发现，以应对 WiFi APEX 中类不在 boot class-loader 的情况。`getScanResults` 返回由 SSID、BSSID、RSSI、频率和 capabilities 构造的 `ScanResult` 列表；`getConnectionInfo` 返回对应连接信息。对 Oplus 扩展字段如 `information elements` 做非空初始化，避免系统消费者对空数组解引用。BLE 测试数据按地址去重，使用 `ScanResult(BluetoothDevice, ScanRecord, rssi, timestamp)` 构造广播结果，原始广播字

节优先，否则按名称生成最小广播记录。蓝牙栈 Hook 根据 Android 15/16 的扫描入口差异选择候选方法，并在无法解析回调时放行真实扫描。

5.4 Sensor Native 适配

JADX 结果显示目标 ROM 的传感器 Java 外壳为 `com.android.server.sensors.SensorService`，持有 native 指针 `mPtr`，通过 `SensorManagerInternal.LocalService` 暴露 Runtime Sensor 的创建、删除和事件发送接口。其 `sendRuntimeSensorEventNative(long,int,int,long,float[])` 只对服务注册的 runtime handle 有效；对真实传感器 handle 调用成功并不等价于事件已进入普通应用连接。项目已有记录据此将 Java 通道作为探测/回退路径，并把全局研究重点放在 `libsensorservice.so` 的分发汇聚点。

Native 方案在编译期按目标 ROM 的函数特征校验入口，在 `SensorService::sendEventsToAllClients` 处读取服务事件缓冲，重写事件值并尾跳原函数；`SensorEventQueue::write` 和 `BitTube::sendObjects` 是保留的兜底入口。该路径需要严格的 ARM64 指令、PAC、内存保护和容量边界处理，安装失败时不修改原代码。论文将其作为系统级测试适配的实现路径；其是否对未配置作用域的测试进程稳定送达，仍应以 `VirEnvDetector` 的事件计数和系统日志进行运行时确认。

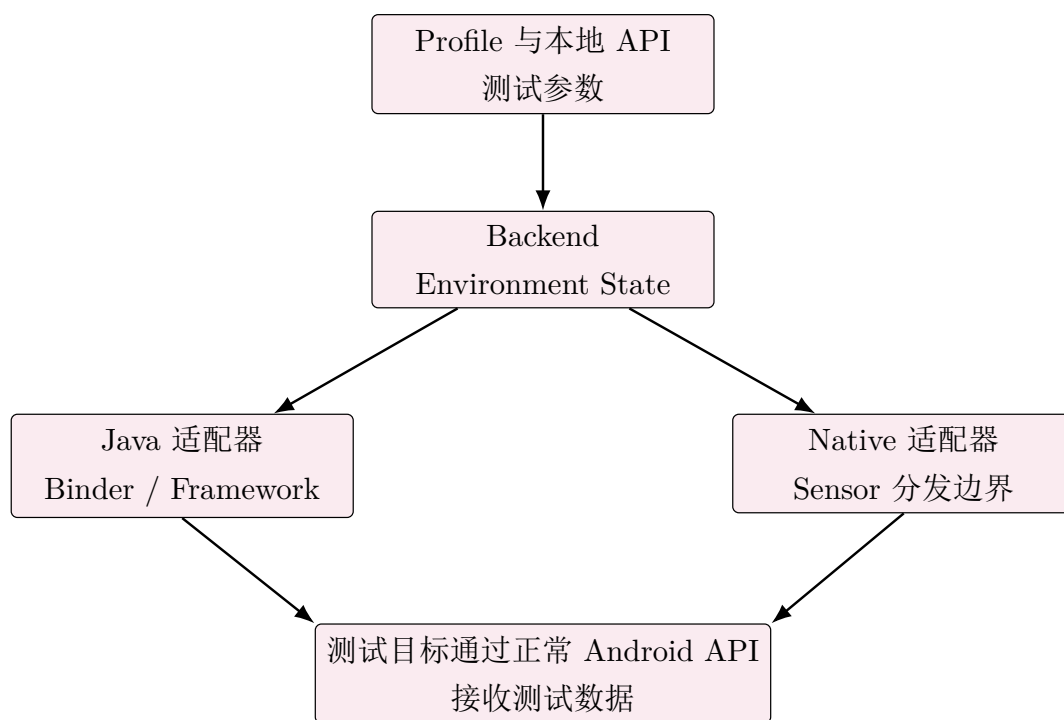


图 3: 测试数据的系统适配流程

六、 实验与结果

6.1 实验方法

本文不重新执行真机测试，而复用项目已有的源码、逆向记录和 `VirEnvDetector` [3] 设计作为实验材料。验证原则是独立检测器优先，并结合模块 Hook 状态、logcat、系统调用结果和检测器报告；截图不作为主要证据。检测器从普通应用视角注册位置、GNSS Status/NMEA、传

感器、BLE、WiFi 和电话读取，并通过本地 API 获取期望配置，形成 PASS、FAIL、SYNCING 或 NOT_ENABLED 判定。结果解释区分“接口回调到达”“数据字段一致”和“Native 全局通道实际送达”三个层级。

6.2 源码与静态证据

源码确认以下能力已形成明确实现链路：单点位置由 `SinglePointLocationEngine` 管理，路线由惰性时间推进的 `RouteEngine` 管理；GNSS 由 `GNSSSimulationEngine`、`OrbitCalculator`、`CoordinateTransform` 和 `SignalModel` 组合生成；`CellInfo`、`WiFi`、`BLE`、`SIM` 由对应工厂和系统服务适配器构造。运动数据由 `VirtualMotionEngine` 统一生成，传感器注入层仅负责投递 [4]。

6.3 已有运行时结论

项目记录显示，目标 Android 15/Oplus 设备上的应用兼容传感器链路能够使 `VirEnvDetector` 观察到步数递增，以及由统一运动模型产生的加速度、重力和磁场测试值；注册真实传感器的路径在模拟激活时被屏蔽，避免测试值与真实事件叠加。GNSS 记录显示，虚拟状态包含卫星数量、星座、SVID、角度、CN0、星历和 `UsedInFix` 字段，且 NMEA 与虚拟位置使用同一 Backend 位置源。

对系统级 `SensorService` Java 通道，已有记录明确指出 Oplus 15 的 Runtime Sensor 事件调用可以返回成功并更新缓存，但对真实传感器 handle 未进入普通应用连接；因此不能将该返回值当作全局送达证据。Native 汇聚点方案提供了更接近真实分发路径的实现，但必须用 detector 的 `events received` 增长、`dumpsys sensorservice`、Hook 计数和 logcat 共同确认。该证据边界使论文不把设计预留写成已经完成的全局能力。

6.4 兼容性结论

兼容矩阵将 Android 15 多项能力标记为 VERIFIED，将 Android 16 多项类和方法签名标记为 STATIC_VERIFIED，并将 WiFi 动态类、前台服务规则及运行时 SIM 链路保留为 REQUIRES_DEVICE。Android 17 及以上为 NOT_ADAPTED。由此，当前结论是：在已验证的软件测试环境中表现稳定；跨版本可通过 Profile 维持候选匹配和回退，但不能由静态签名推导真机运行成功。

七、 讨论与限制

7.1 软件层边界

框架生成的是 Android API 可观察的数据，不是 GNSS 射频信号。GNSS 轨道采用圆轨道近似，未模拟真实星历、伪距、载波相位、多路径、接收机时钟误差、导航电文和射频前端；因此不能替代硬件在环或射频仿真器。类似地，WiFi、BLE 和 `CellInfo` 是结构化测试对象，不会改变物理网络、无线电环境或基带状态。

7.2 厂商 ROM 差异

系统服务可能移动到 APEX、厂商 JAR 或独立 APK，R8 可能重命名内部方法，Binder 参数也可能随版本变化。Native Hook 还依赖函数布局、指令特征、PAC 和可执行内存权限。项

目通过 Profile、多候选匹配、二进制特征检查和 fail-open 降低风险，但这不能保证未知 ROM 兼容。任何涉及 system_server 的新适配都应先静态分析，再在独立检测器上做最小运行验证。

7.3 数据一致性与可观测性

环境多源一致性依赖统一时间轴。若适配器缓存滞后、监听器生命周期处理错误或某一数据面继续返回真实值，融合应用仍可能观察到残差。因此测试报告应记录配置版本、快照时间、Hook 状态、回调时间、原始/虚拟摘要和系统日志，而不能只记录 UI 文本。对于事件驱动计步器，静止状态没有真实事件，必须区分“没有事件”与“值为零”，并在需要时使用主动事件机制。

7.4 后续方向

后续工作包括：为 Android 14、16 及后续版本补充可复现实机证据；建立不依赖具体 ROM 偏移的 Sensor HAL 或系统扩展适配；将路线速度与统一运动状态双向约束；加入更严格的时间戳、单调性和跨源一致性指标；并把环境包回放与 CI 测试用例结合。上述方向均属于测试基础设施增强，不改变项目不针对第三方应用进程开发的架构约束。

八、 总结

本文从问题背景、系统架构、数学模型、Android 实现和实验边界五个方面分析了 ZhangVirtualEnv。框架以 Backend Core 维护统一环境状态，以系统服务和 Framework 边界的适配器将状态转换为 Android API 数据，并通过 Profile 隔离 Android 版本与厂商 ROM 差异。路线引擎实现按时间推进的单点和折线轨迹，GNSS 引擎实现固定卫星目录、连续轨道近似、WGS-84 坐标变换、ENU 几何和连续 C/N0；运动引擎以统一步态相位生成步数、加速度、重力和陀螺仪数据；CellInfo、WiFi、BLE 和 SIM 适配器则通过版本兼容构造器和 Binder 服务边界提供结构化测试输入。

源码和逆向证据同时表明，数据路径的“可调用”并不等于“已送达”。尤其是 Android 15 Oplus SensorService 的 Runtime Sensor 通道存在真实传感器句柄边界，Native 分发汇聚点虽是合理的系统级研究方向，也必须由 VirEnvDetector、Hook 计数、logcat 与系统调用共同证明。因此，本文最终将 ZhangVirtualEnv 定位为可重复的 Android 系统环境参数测试与兼容性验证框架，而非真实物理环境或射频信号仿真器。

参考文献

- [1] Android Open Source Project. Android location and gnss framework apis, 2025. Android platform documentation and framework source.
- [2] Android Open Source Project. Android sensor framework and sensorservice, 2025. Android platform documentation and framework source.
- [3] FairyXH. Virendetector: Independent android environment test validator, 2026. Companion validation project.
- [4] FairyXH. Zhangvirtualenv: Android system environment parameter testing framework, 2026. Project source code, reverse-engineering notes and compatibility records.
- [5] National Geospatial-Intelligence Agency. Department of defense world geodetic system 1984, 2014. WGS 84 reference specification.